

Testes de Software

Análise de Eficácia de Testes com Teste de Mutação

Júlia de Souza Ventura

Matrícula: 869002

1. Análise Inicial

A cobertura do código inicial, obtida através do `npm test -- --coverage`, mostrou números altos:

```
-----|-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----|
All files | 85.41   | 58.82    | 100     | 98.64   |
operacoes.js | 85.41   | 58.82    | 100     | 98.64   | 112
-----|-----|-----|-----|-----|
Test Suites: 1 passed, 1 total
Tests:       50 passed, 50 total
Snapshots:   0 total
Time:        0.286 s, estimated 1 s
Ran all test suites.
Process finished with exit code 0
```

Statements: 85,41% | Brancher: 58,82% | Function: 100% | Lines: 98,64%

Em seguida, o StrykerJS foi configurado e executado pela primeira vez, apresentando um resultado diferente:

```
Ran 1.19 tests per mutant on average.
-----|-----|-----|-----|-----|
File      | % Mutation score | % Mutation score | # killed | # timeout | # survived | # no cov | # errors
-----|-----|-----|-----|-----|-----|
All files | 73.71 | 78.11 | 154 | 3 | 44 | 12 | 0
operacoes.js | 73.71 | 78.11 | 154 | 3 | 44 | 12 | 0
-----|-----|-----|-----|-----|
```

Mutation Score: 73,71% | Mortos: 154 | Sobreviventes: 44 | Sem cobertura: 12

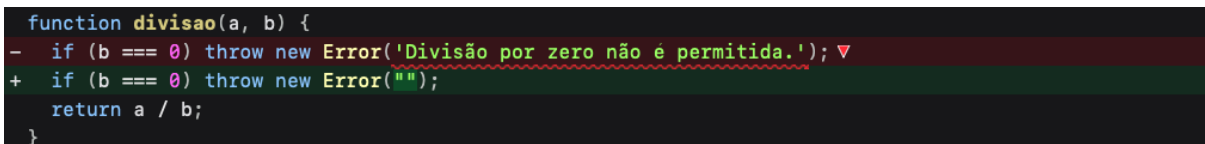
Conseguimos perceber que apesar de 98,64% das linhas terem sido cobertas, a pontuação de mutação, ou seja, o percentual de mutantes que os testes conseguiram detectar foi de apenas 73,71%. O teste de mutação revelou que a cobertura mede se o código foi executado e não se ele foi verificado, expondo 44 mutantes que a suíte original não conseguia detectar.

2. Análise de Mutantes Críticos

❖ Mutante 1 - Mensagem de erro em divisão()

O Stryker substituiu a mensagem do erro por uma string vazia:

```
function divisao(a, b) {
-   if (b === 0) throw new Error('Divisão por zero não é permitida.');
```



```
+   if (b === 0) throw new Error("");
    return a / b;
}
```

O teste original usava `.toThrow()` sem argumento, que verifica apenas se algum erro foi lançado, não verificando a mensagem. Como o mutante ainda lança um erro, o teste passou normalmente.

❖ Mutante 2 - Condição removida em raizQuadrada()

```
function raizQuadrada(n) {  
- if (n < 0) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');
```


+ if (false) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');
 return Math.sqrt(n);
}

```
function raizQuadrada(n) {  
- if (n < 0) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');
```


+ if (n <= 0) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');
 return Math.sqrt(n);
}

Com `if (false)`, nenhum erro seria lançado para números negativos. Além disso, um segundo mutante trocou `n < 0` por `n <= 0`, e como o teste nunca passava o valor 0, esse caso também passava despercebido. Em ambos os casos, o teste só usava o valor 16 (positivo), deixando os cenários críticos sem cobertura.

❖ Mutante 3 - Operador alterado em isPrimo()

```
function isPrimo(n) {  
- if (n <= 1) return false;
```


+ if (n < 1) return false;

Com `n < 1`, o número 1 não é mais barrado e entra no loop, fazendo a função retornar `true` incorretamente. O teste só usava `isPrimo(7)` e nunca testava o valor 1.

3. Solução Implementada

Para cada mutante analisado, novos casos de teste foram adicionados:

❖ Mutante 1 - divisao()

```
test('4. deve dividir e lançar erro para divisão por zero', () :void => {  
  expect(divisao( a: 10, b: 2)).toBe( expected: 5);  
  expect(() => divisao( a: 5, b: 0)).toThrow( expected: 'Divisão por zero não é permitida.');
```


});

Ao passar a mensagem esperada para `.toThrow()`, o teste passa a verificar não somente se um erro foi lançado mas também se a mensagem é exatamente a esperada.

❖ Mutante 2 - raizQuadrada()

```
test('6. deve calcular a raiz quadrada de um quadrado perfeito e lançar erro para números negativos', () :void => {  
  expect(raizQuadrada( n: 16)).toBe( expected: 4);  
  expect(raizQuadrada( n: 0)).toBe( expected: 0);  
  expect(() :any | number => raizQuadrada( n: -5)).toThrow( expected: 'Não é possível calcular a raiz quadrada de um número negativo.');
```


});

Para cobrir ambos os casos, dois novos testes foram adicionados: um com valor -5, que ativa o bloco de erro e mata o mutante `if(false)`, e outro com o valor 0, que garante que a função retorna 0 corretamente e mata o mutante `n <= 0`.

❖ Mutante 3 - isPrimo()

```
test('33. deve verificar que um número é primo', () :void => {  
  expect(isPrimo( n: 7)).toBe( expected: true);  
  expect(isPrimo( n: 4)).toBe( expected: false);  
  expect(isPrimo( n: 1)).toBe( expected: false);  
});
```

Testar o valor 1 mata o mutante $n < 1$. Testar um número não primo como 4 mata os mutantes que alteram o loop inteiro.

4. Resultados Finais

Após as melhorias, o StrykerJS foi executado novamente:

```
Ran 1.24 tests per mutant on average.  
-----|-----|-----|-----|-----|-----|  
File      | % Mutation score |  
-----|-----|-----|-----|-----|  
total | covered | # killed | # timeout | # survived | # no cov | # errors |  
-----|-----|-----|-----|-----|  
All files | 98.12 | 98.12 | 206 | 3 | 4 | 0 | 0 |  
operacoes.js | 98.12 | 98.12 | 206 | 3 | 4 | 0 | 0 |  
-----|-----|-----|-----|-----|
```

Mutation Score: 98,12% | Mortos: 206 | Sobreviventes: 4 (todos equivalentes)

A pontuação subiu de 73,71% para 98,12%. Os 4 mutantes restantes são equivalentes e não podem ser eliminados por testes, pois representam alterações no código que não mudam o comportamento observável do programa em nenhum cenário possível.

Além disso, todos os 50 testes continuaram passando com o código original após as melhorias, confirmando que os novos casos de teste são corretos e não introduziram falsos negativos na suíte.

5. Conclusão

Este trabalho mostrou que cobertura de código não garante qualidade de testes. Com 98% de linhas cobertas, a suíte original ainda deixava passar 44 alterações no código que os testes não conseguiram detectar. O teste de mutação foi essencial para identificar as fraquezas reais da suíte: testes que não verificavam mensagens de erro, que não testavam valores-limites como zero e um, e que só cobriam casos positivos sem testar cenários de falha. Corrigir esses pontos foi o que permitiu elevar a pontuação de mutação de 73,71% para 98,12%, provando que uma suíte de testes só cumpre seu papel quando verifica não apenas se o código roda, mas se ele se comporta corretamente em todos os cenários possíveis.